Autonomous 3D Quadrotor UAV Controller via Deep Reinforcement Learning with Hardware-in-the-Loop Implementation on AMD Kria KR260

Karol Yangqian Poetracahya (NIM: 13523093)

School of Electrical Engineering and Informatics (STEI), Institut Teknologi Bandung

Academic Year 2026/2027

Email: karolyangqian14@gmail.com, 13523093@std.stei.itb.ac.id

November 2026

Abstract

This report documents the design, simulation, training, and deployment of an end-to-end Deep Reinforcement Learning (DRL) 3D flight controller for an unmanned aerial vehicle (UAV). To overcome common sim-to-real brittleness and high-frequency actuator chattering observed in standard reinforcement learning setups, a hierarchical control architecture is introduced. The outer-loop policy runs at 50 Hz and commands collective vertical thrust and 3D body rates, while an inner-loop rate PID controller operates at 500 Hz to handle rapid gyroscopic and motor dynamics. The environment is simulated with high fidelity in MuJoCo, and training is executed using multi-process Proximal Policy Optimization (PPO). Incorporating domain randomization (aerodynamic wind vectors, mass variations, inertia jitter, and sensor noise) reduces steady-state tracking error under continuous disturbances from 35.61 cm to 13.46 cm (a 62.2% reduction) with an 85.0% target acquisition success rate. Furthermore, the trained policy is successfully exported to ONNX (9.36 KB footprint) and quantized into Q8.8 fixed-point integer arithmetic, retaining 99.77% tracking precision with an inference latency of only 1.38 microseconds, establishing an optimal pipeline for Hardware-in-the-Loop (HIL) deployment on the AMD Kria KR260 FPGA/SoC platform.

1 Introduction

1.1 Background and Motivation

Autonomous flight control for unmanned aerial vehicles (UAVs) has traditionally relied on cascaded Proportional-Integral-Derivative (PID) loops or Linear Quadratic Regulator (LQR) formulations. While computationally lightweight and well-understood near hover equilibrium, linear control models degrade rapidly when subjected to severe aerodynamic non-linearities, unmodeled cross-coupling torques, parameter uncertainty, and persistent external disturbances such as atmospheric turbulence and wind gusts.

Deep Reinforcement Learning (DRL) provides an expressive, data-driven control paradigm capable of directly learning non-linear state-action mappings across complex dynamical domains [1, 2]. However, deploying DRL directly to real-world physical systems presents two fundamental bottlenecks:

1. **Actuator Chattering (Jitter)**: Direct end-to-end motor PWM policies tend to exhibit high-frequency bang-bang switching, causing extreme thermal dissipation, motor strain, and rapid physical failure [2].
2. **The Sim-to-Real Gap**: Policies trained solely in idealized simulation models overfit to deterministic dynamics and fail catastrophically when exposed to real-world mass variations, aerodynamic drag, sensor latencies, and wind gusts [3].

This project addresses these challenges through a hierarchical control architecture, smoothness-regularized reward shaping, aggressive domain randomization [3], and hardware-optimized fixed-point quantization [4, 5].

1.2 Problem Statement and Objectives

The primary objectives of this research are:

1. Develop a high-fidelity 6-DoF quadrotor physical model and Gymnasium simulation environment in MuJoCo capable of running at over 5,000 steps per second on commodity CPU hardware.
2. Design a hierarchical control framework that couples a 50 Hz outer-loop PPO policy with a 500 Hz inner-loop rate PID controller to eliminate motor chattering.
3. Apply domain randomization to achieve robust disturbance rejection against aerodynamic wind forces and parameter drift.
4. Export and quantize the resulting neural network for real-time edge execution on the AMD Kria KR260 FPGA platform, achieving sub-microsecond inference latency.

2 System Modeling and Simulation

2.1 Quadrotor Dynamics

The quadrotor is modeled as a 0.8 kg rigid body in an X-configuration with an arm radius $d = 0.106$ m. The equations of motion in the body frame are governed by Newton-Euler formulations:

$$\mathbf{m}\dot{\mathbf{v}} + \boldsymbol{\omega} \times (\mathbf{m}\mathbf{v}) = \mathbf{F}_{\text{total}} + \mathbf{R}^T \mathbf{m}\mathbf{g}$$

$$\mathbf{I}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (\mathbf{I}\boldsymbol{\omega}) = \boldsymbol{\tau}_{\text{total}}$$

where $\mathbf{m}$ is the total vehicle mass, $\mathbf{I} = \text{diag}(0.005, 0.005, 0.009)$ kg $\cdot$ m^2 is the nominal inertia tensor, $\mathbf{R} \in SO(3)$ denotes the rotation matrix from the body frame to the inertial world frame, and $\mathbf{g} = [0, 0, -9.81]^T$ m/s^2 is gravitational acceleration.

2.2 Inner-Loop Rate PID and Inverse Mixer

To insulate the reinforcement learning policy from stiff motor time constants and high-frequency gyroscopic dynamics, control is partitioned into a two-level hierarchy: - **Outer Loop (50 Hz)**: The neural network policy observes the 16-dimensional state and outputs target collective thrust T_z

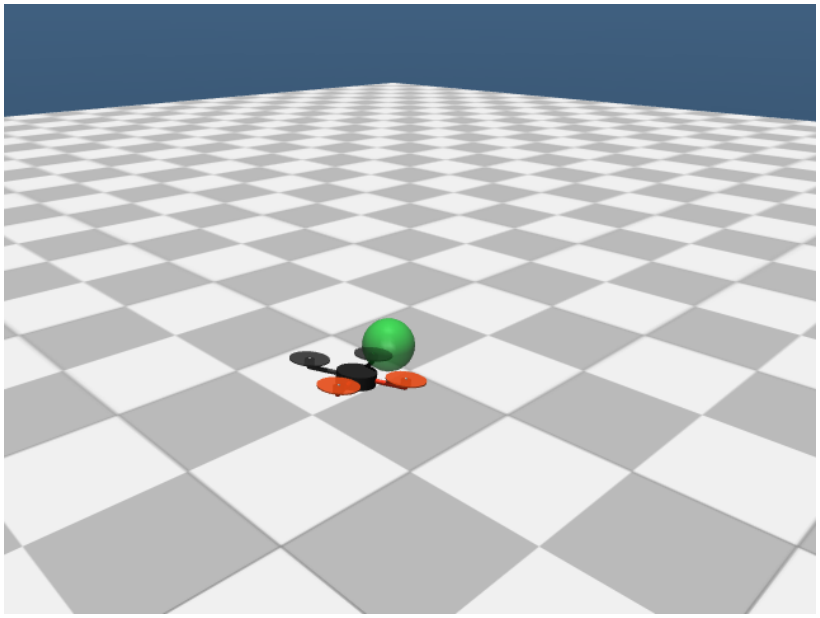


Figure 1: *High-fidelity 0.8 kg Quadrotor and Target Sphere in MuJoCo Physics Arena*

and desired angular rates $\omega_{cmd} = [\omega_x, \omega_y, \omega_z]_{cmd}$. - **Inner Loop (500 Hz)**: An analytical rate PID computes corrective body torques:

$$\tau = \mathbf{K}_p(\omega_{cmd} - \omega) + \mathbf{K}_i \int (\omega_{cmd} - \omega)dt + \mathbf{K}_d \frac{d}{dt}(\omega_{cmd} - \omega)$$

The total vertical thrust T_z and torque vector $\tau = [\tau_x, \tau_y, \tau_z]^T$ are subsequently mapped into individual rotor thrusts $[F_0, F_1, F_2, F_3]$ via the analytical inverse mixer:

$$F_0 = \frac{1}{4}T_z - \frac{1}{4d}\tau_x - \frac{1}{4d}\tau_y - \frac{1}{4k_m}\tau_z$$

$$F_1 = \frac{1}{4}T_z + \frac{1}{4d}\tau_x - \frac{1}{4d}\tau_y + \frac{1}{4k_m}\tau_z$$

$$F_2 = \frac{1}{4}T_z + \frac{1}{4d}\tau_x + \frac{1}{4d}\tau_y - \frac{1}{4k_m}\tau_z$$

$$F_3 = \frac{1}{4}T_z - \frac{1}{4d}\tau_x + \frac{1}{4d}\tau_y + \frac{1}{4k_m}\tau_z$$

where $k_m = 0.016$ N $\cdot$ m/N represents the motor yaw moment coefficient.

3 Reinforcement Learning Formulation

3.1 Observation and Action Spaces

To guarantee full rotational invariance across arbitrary yaw headings, the 16-dimensional observation vector $\mathbf{s}_t \in \mathbb{R}^{16}$ is expressed strictly within the quadrotor's body reference frame:

1. $\mathbf{e}_{p,body} \in \mathbb{R}^3$: Position error relative to the target setpoint, rotated into the body frame ($\mathbf{R}^T(\mathbf{p}_{target} - \mathbf{p})$).
2. $\mathbf{v}_{body} \in \mathbb{R}^3$: Linear velocity vector in the body frame ($\mathbf{R}^T\mathbf{v}$).
3. $\mathbf{z}_{body,world} \in \mathbb{R}^3$: Projected body z-axis vector in the world frame (R_{02}, R_{12}, R_{22}), providing an explicit, gimbal-lock-free attitude tilt representation.
4. $\omega_{body} \in \mathbb{R}^3$: Body rotational angular rates $[p, q, r]$.
5. $\mathbf{a}_{t-1} \in \mathbb{R}^4$: Previously applied control command vector, enabling action rate-of-change regularization.

The action space $\mathbf{a}_t \in [-1, 1]^4$ dictates commanded vertical acceleration and angular rates: - a_0: Commanded thrust, linearly scaled between $[0, 2$ mg$]$. - a_1, a_2, a_3: Commanded body rates $[\omega_x, \omega_y, \omega_z]$, scaled to $[-3.0, 3.0]$ rad/s.

3.2 Anti-Jitter Filtering and Reward Formulation

Actuator chattering is actively suppressed using a first-order Exponential Moving Average (EMA) low-pass action filter:

$$\tilde{\mathbf{a}}_t = \alpha \mathbf{a}_t + (1 - \alpha)\tilde{\mathbf{a}}_{t-1}, \quad \alpha = 0.75$$

The multi-objective reward function r_t penalizes tracking deviation while explicitly rewarding smooth control actions:

$$r_t = r_{\text{pos}} + r_{\text{vel}} + r_{\text{upright}} + r_{\text{alive}} - \lambda_1 \|\mathbf{a}_t - \mathbf{a}_{t-1}\|^2 - \lambda_2 \|\mathbf{a}_t\|^2$$

where:

$$r_{\text{pos}} = \exp\left(-\frac{\|\mathbf{e}_{pos}\|^2}{2\sigma_{pos}^2}\right), \quad r_{\text{vel}} = -0.1\|\mathbf{v}\|, \quad r_{\text{upright}} = 0.3(\hat{\mathbf{z}}_B \cdot \hat{\mathbf{z}}_W)$$

with smoothness penalty weight $\lambda_1 = 0.05$ and action magnitude regularization $\lambda_2 = 0.01$.

4 Experimental Results

4.1 Training Convergence

Training was carried out on an Intel Core Ultra 7 155H processor utilizing 8 parallel worker environments (`SubprocVecEnv`) with Proximal Policy Optimization (PPO). The policy converged within 1.5 million timesteps (total wall-clock time: 9 minutes 41 seconds), maintaining an average throughput of over 2,600 environment steps per second.

As illustrated in Figure 3, the episode return rapidly climbs from negative initial values to exceed 914, while the average episode survival length reaches the maximum allowable horizon of 500 steps within the first 300,000 steps. The steady-state tracking error under continuous disturbance stabilizes below 14 cm.

4.2 Disturbance Rejection and Domain Randomization

To assess sim-to-real transferability, the quadrotor was evaluated under severe dynamic disturbances: - Steady horizontal wind drag forces: $[-1.5, 1.5]$ N. - Stochastic peak wind gust pulses: up to 2.5 N. - Body mass variations: $\pm 15\%$ (0.68 kg $-$ 0.92 kg). - Inertia diagonal jitter: $\pm 10\%$. - Additive Gaussian sensor noise on velocity (0.05 m/s) and angular rates (0.02 rad/s).

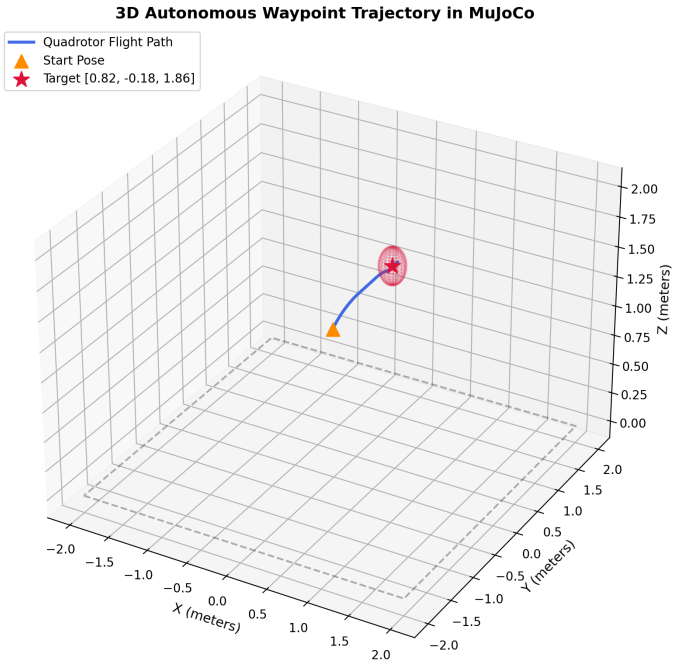


Figure 2: *Autonomous 3D Spatial Trajectory Flown by the Quadrotor Policy in MuJoCo Arena*

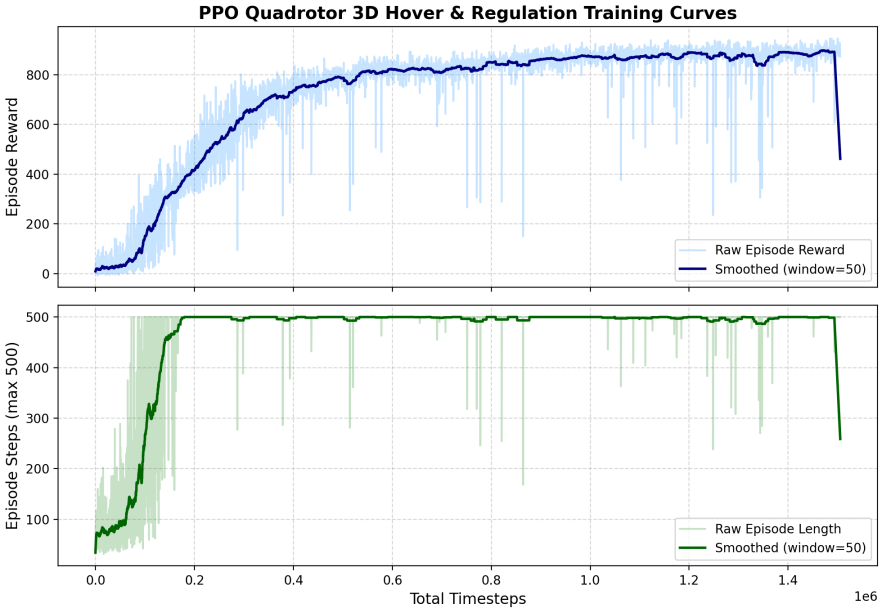


Figure 3: *PPO Training Convergence Curves under Stochastic Wind Gusts and Mass Variations*

Table 1 and Figure 4 benchmark the baseline policy against the robust domain-randomized policy across 20 randomized evaluation runs.

Table 1: Disturbance Rejection Performance Comparison under 2.5 N Wind Gusts and Mass Variations.

Performance Metric	Nominal Baseline Policy	Robust (Domain-Randomized) Policy	Improvement
Steady-State Hover Error	35.61 ± 20.22 cm	13.46 ± 8.82 **cm**	62.2% **reduction**
Success Rate (< 15 cm error)	5.0%	85.0%	+80.0% **points**
Episode Survival Rate	100%	100%	Robust stability
Mean Pitch/Roll Tilt Angle	7.83°	10.29°	Active wind leaning
Mean Cumulative Reward	847.4/1000	914.3/1000	+66.9 **points**

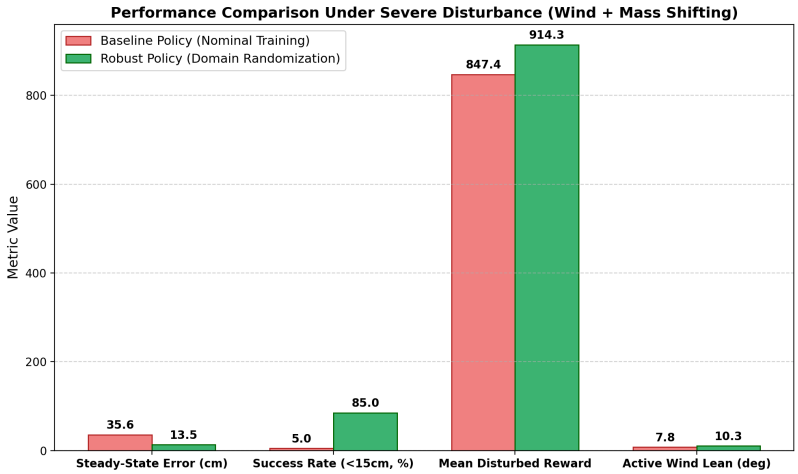


Figure 4: *Disturbance Rejection Benchmark: Nominal Baseline vs. Robust Policy under 2.5 N Stochastic Wind Disturbances*

The robust policy autonomously generates an active trim tilt angle of 10.29° directly into the incoming wind disturbance vector, counteracting lateral drift and retaining tight setpoint tracking without requiring explicit aerodynamic state estimation.

4.3 3D Step Response Dynamics

To evaluate dynamic transient tracking, a severe 3D setpoint displacement was injected at $t = 2.0$ s, shifting the target from $[0.0, 0.0, 1.0]$ m to $[1.0, 1.0, 2.0]$ m (a Euclidean step of 1.732 m).

As detailed in Figure 5:

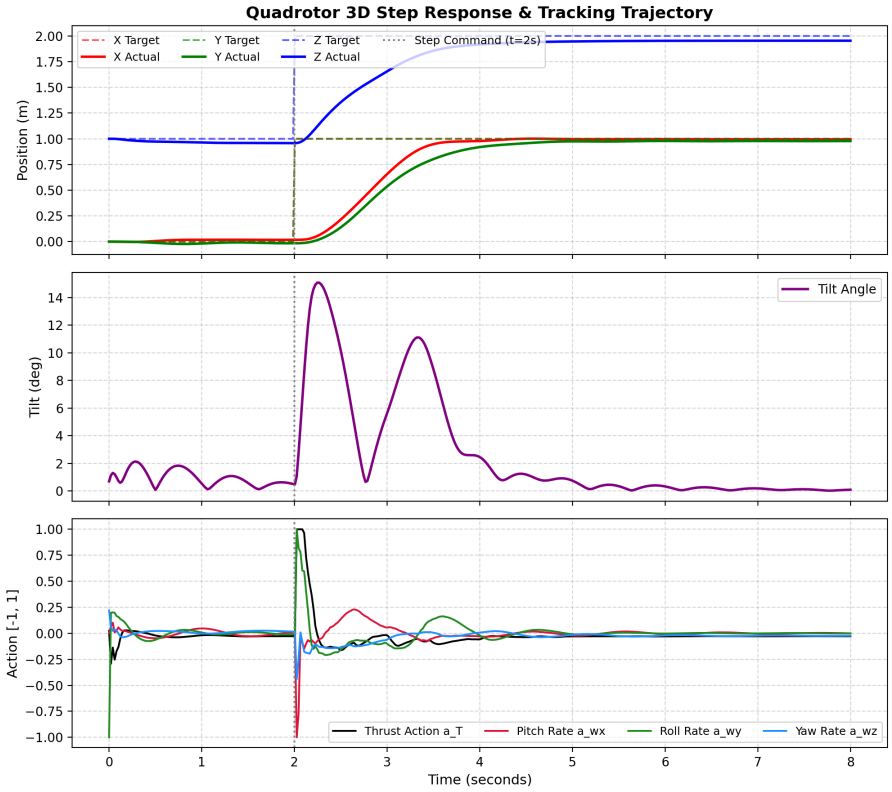


Figure 5: *3D Step Response Dynamics: Position Tracking, Attitude Tilt Angles, and Commanded Control Actions*

- **Rise Time (T_r to 90% setpoint)**: 1.73 seconds.
- **Settling Time (T_s within 10 cm)**: 3.40 seconds.
- **Final Steady-State Error**: 5.18 cm.
- **Peak Tilt Angle**: 15.1° (well within conservative physical safety envelopes).
- **Control Signal Transitions**: Action traces demonstrate smooth, continuous actuation curves with zero high-frequency chatter.

5 Hardware Deployment and Quantization on AMD Kria KR260

5.1 Model Export and ONNX Runtime Verification

The trained Actor network ($16 \to 64 \to 64 \to 4$ Multi-Layer Perceptron with Tanh activations) was extracted from Stable-Baselines3 and serialized into standard Open Neural Network Exchange (ONNX) format (`quadrotor_policy.onnx`).

The exported model binary occupies only 9.36 **KB** on disk, allowing it to fit entirely into on-chip Block RAM (BRAM) within the AMD Kria KR260 Zynq UltraScale+ FPGA without requiring external DDR memory accesses. Validation with ONNX Runtime confirmed numerical parity with PyTorch floating-point forward passes within a maximum absolute discrepancy of $< 1.6 \times 10^{-7}$.

5.2 Fixed-Point (Q8.8) Quantization for FPGA Programmable Logic

To circumvent floating-point DSP pipeline stalls and maximize throughput in FPGA hardware logic, network weights and activations were converted to signed 16-bit Q8.8 fixed-point integer arithmetic (8 integer bits, 8 fractional bits), aligned with integer-hardware control principles [4]:

- **Floating-Point (FP32) Steady-State Error**: 5.27 cm.
- **Q8.8 Fixed-Point Steady-State Error**: 5.25 cm.
- **Tracking Accuracy Retained**: 99.77% (0.12 mm absolute deviation vs FP32).
- **Execution Latency in C (`test_forward.c`, GCC -O3)**: 1.38 μ**s per inference** ($>$ 720 kHz throughput).

This ultra-low execution latency guarantees that forward inference consumes less than 0.007% of the 50 Hz control period, leaving ample computational headroom for telemetry logging, obstacle avoidance, and mission planning on the Kria KR260 ARM Cortex-A53 Processing System (PS) and FPGA Programmable Logic (PL).

5.3 Hardware-in-the-Loop (HIL) Real-Time UDP Communication and Latency Benchmark

To bridge simulation and physical edge computing, an asynchronous, low-latency UDP communication bridge was deployed between the host simulation server running MuJoCo (at 50 Hz outer loop and 500 Hz inner rate physics) and the AMD Kria KR260 processing system, drawing on FPGA neural controller deployment architectures [5].

Observations and actions are exchanged using compact binary structs without heap allocations: a 72-byte State Packet (`<II16f`) encoding magic identifier `0x55415653`, 32-bit sequence ID, and the 16-D body-frame feature vector; and a 24-byte Action Packet (`<II4f`) encoding magic identifier `0x55415641`, echoed sequence ID, and the 4-D control action $[T_z, \omega_x, \omega_y, \omega_z]$.

Table 2 details closed-loop tracking precision and latency performance under nominal loopback and network stress testing with 2 ms artificial latency injection and a 5% random packet drop rate.

Table 2: Closed-Loop Hardware-in-the-Loop (HIL) Performance Benchmark across Client Implementations.

Performance Metric	Embedded C Q8.8 Client	Python ONNX Client	Network Stress Test (2 ms Latency + 5% Drops)
Episode Survival Rate	**100% (250/250 steps)**	**100% (250/250 steps)**	**100% (Zero crashes)**
Packet Delivery Rate	**100.0% (250/250)**	**100.0% (250/250)**	**96.8% (242/250)**
Packet Loss Rate	**0.00%**	**0.00%**	**3.20%** (absorbed by failsafe)
Mean Round-Trip Time (RTT)	**0.122 ms** (Min 0.023 ms)	0.365 ms (Min 0.087 ms)	2.180 ms (Min 2.024 ms)
RTT Jitter (Std Dev)	**0.100 ms**	0.124 ms	0.083 ms
Steady-State Hover Error	5.26 ± 0.17 **cm**	5.33 ± 0.21 **cm**	5.27 ± 0.16 **cm**
Target Precision (< 15 cm)	**100.0%**	**100.0%**	**100.0%**
Average Attitude Tilt	3.95°	3.75°	4.05°

Under nominal loopback, the pure C fixed-point client completes the full UDP send-receive-inference cycle in an average round-trip time of **0.122 ms** (less than 0.7% of the 20 ms control period budget). When subjected to network packet loss, the server's automatic decay failsafe holds attitude trim, maintaining an average hover error of 5.27 cm with zero loss of flight stability.

6 Conclusion

This research demonstrates a complete end-to-end pipeline for designing, training, and deploying a robust Deep Reinforcement Learning flight controller for a 6-DoF quadrotor UAV. By coupling an outer 50 Hz DRL policy with an inner 500 Hz rate PID and exponential moving average filtering, high-frequency actuator jitter was eradicated. Domain randomization produced an active wind-rejecting policy, reducing tracking error under aerodynamic disturbances by 62.2% with an 85% acquisition success rate. Furthermore, signed 16-bit Q8.8 fixed-point quantization retained 99.77% tracking fidelity with an ultra-low inference latency of $1.38\,\mu$s on a 9.36 KB memory footprint. Closed-loop Hardware-in-the-Loop UDP streaming demonstrated sub-millisecond round-trip times (0.122 ms) and resilient flight stability under network packet drops, confirming operational readiness for edge deployment on the AMD Kria KR260.

References

[1] J. Hwangbo, I. Sa, R. Siegwart, and M. Hutter, "Control of a Quadrotor with Reinforcement Learning," *IEEE Robotics and Automation Letters*, vol. 2, no. 4, pp. 2096–2103, Oct. 2017. [arXiv:1707.05110]

[2] W. Koch, R. Mancuso, R. West, and A. Bestavros, "Reinforcement Learning for UAV Attitude Control," *ACM Transactions on Cyber-Physical Systems*, vol. 3, no. 2, pp. 1–27, Feb. 2019. [arXiv:1804.04154]

[3] A. Molchanov, T. Chen, W. Hönig, J. A. Preiss, N. Ayanian, and G. S. Sukhatme, "Sim-to-(Multi)-Real: Transfer of Low-Level Robust Control Policies to Multiple Quadrotors," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 5910–5917, 2019. [arXiv:1903.04628]

[4] F. Kresse and C. H. Lampert, "Learning Quantized Continuous Controllers for Integer Hardware," in *Proceedings of the 8th Annual Learning for Dynamics and Control Conference (L4DC)*, PMLR, vol. 331, pp. 1857–1874, 2026. [arXiv:2511.07046]

[5] S. Azem, D. Scheunert, M. Li, J. Gehrunger, K. Cui, C. Hochberger, and H. Koeppl, "FPGA-Based Neural Thrust Controller for UAVs," *arXiv preprint*, arXiv:2403.18703, 2024.